

Fenêtres graphiques avec Tkinter

Introduction

- Le module *Tkinter* est inclu dans les distributions Python (pas besoin de faire d'installation de module externe)
- *Tkinter* contrôle la bibliothèque graphique Tk (*Tool Kit*), *Tkinter* signifiant *tk interface*.
- Cette bibliothèque Tk peut être également contrôlée par d'autres langages (Tcl, perl, etc.).
- Il existe beaucoup d'autres modules pour construire des applications graphiques. Par exemple : [wxpython](#), [PyQt](#), [PyGObject](#), turtle (déjà vue), etc
 - [Information Générale](#) et [API](#)

Programmation événementielle

- Le gestionnaire d'événements est un genre de « boucle infinie » qui surveille la moindre action de la part de l'utilisateur. C'est lui qui déclenchera une action lors de l'interaction de l'utilisateur avec des éléments de l'interface graphique (GUI).
- Ce mécanisme de gestion par événements fera appel à des *callback*.
- Une fonction *callback* est une fonction passée en argument d'une autre fonction.

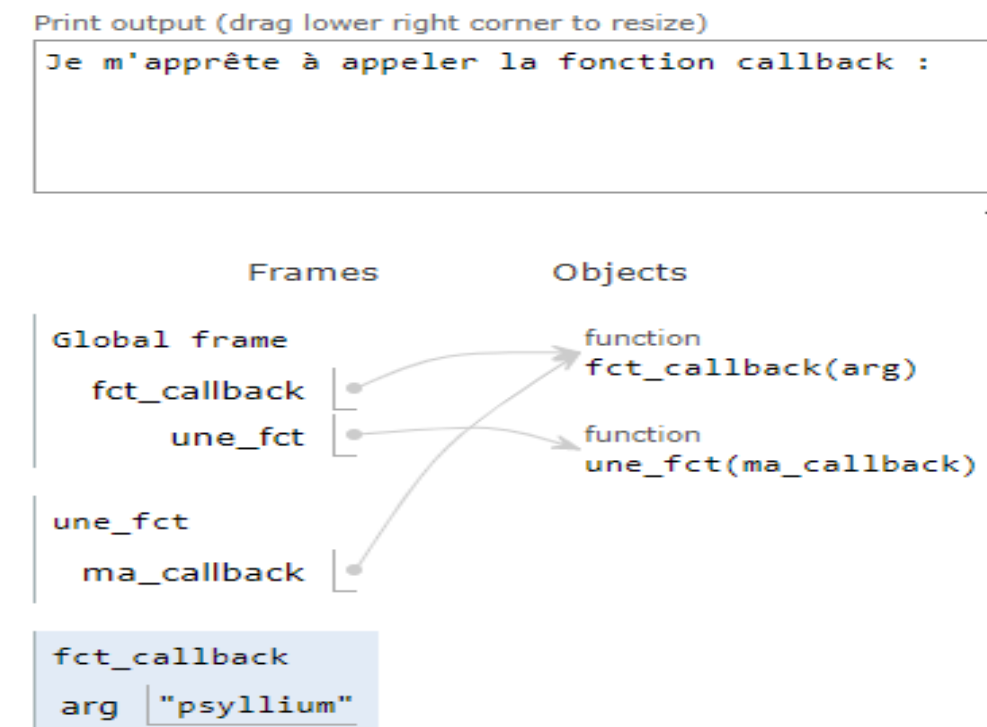
Python 3.6
([known limitations](#))

```
1 #callback.py
2 def fct_callback(arg):
3     print(f"J'aime bien le {arg} !")
4
5
6 def une_fct(ma_callback):
7     print("Je m'apprête à appeler la fonction callback :")
8     ma_callback("psyllium")
9     print("Je termine.")
10
11 if __name__ == "__main__":
12     une_fct(fct_callback)
```

[Edit this code](#)

→ line that just executed
→ next line to execute

<< First < Prev Next > Last >>



Tkinter dans un script

```
1 #baseTkinter.py
2 import tkinter as tk
3
4 racine = tk.Tk()
5 etiquette = tk.Label(racine, text="J'aime Python !")
6 bouton = tk.Button(racine, text="Quitter", command=racine.quit) #quitte la mainloop, ne détruit pas
7 bouton["fg"] = "red" #autre façon de modifier les paramètres d'un widget, a posteriori
8 etiquette.pack()
9 bouton.pack()
10 racine.mainloop() #gestion événementielle
11 print("C'est terminé !") #ne s'affichera que lorsqu'on quittera la boucle principale
12
```



- Une fois l'appel à la boucle principale *racine.mainloop()*, la fenêtre apparaît
- Notez le callback à la fonction *quit()* dans les arguments du bouton
- La fenêtre n'est pas détruite à la fin! Il faudrait changer quit pour destroy.

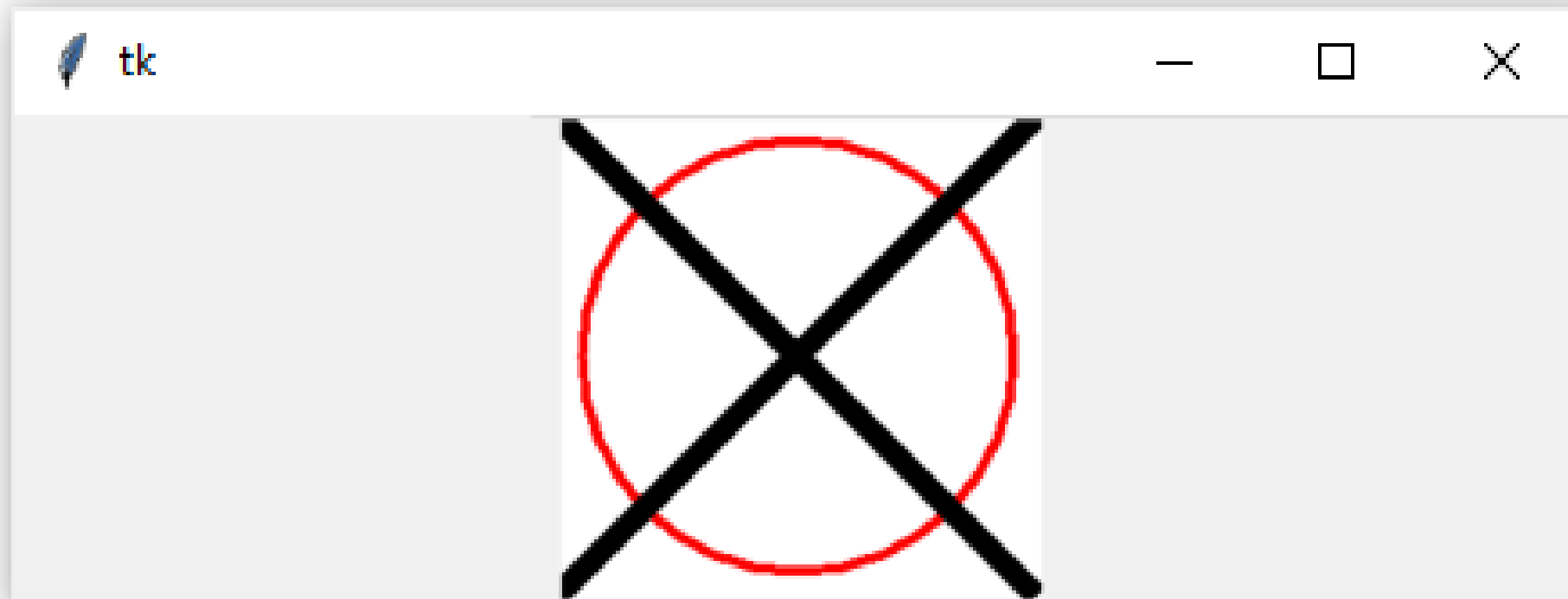
Tkinter avec une classe

```
1 #classeTkinter.py
2 import tkinter as tk
3
4 class Application(tk.Tk):
5     def __init__(self):
6         tk.Tk.__init__(self)
7         self.creer_widgets() #on crée et rattache les widgets en même temps que l'application
8
9     def creer_widgets(self):
10        self.etiquette = tk.Label(self, text="J'aime Python !")
11        self.bouton = tk.Button(self, text="Quitter", fg = "red", command=self.quit) # callback à quit()
12        self.etiquette.pack()
13        self.bouton.pack()
14
15
16 if __name__ == "__main__":
17     app = Application()
18     app.title("Ma Première Application Tkinter")
19     app.mainloop()
20
21
```



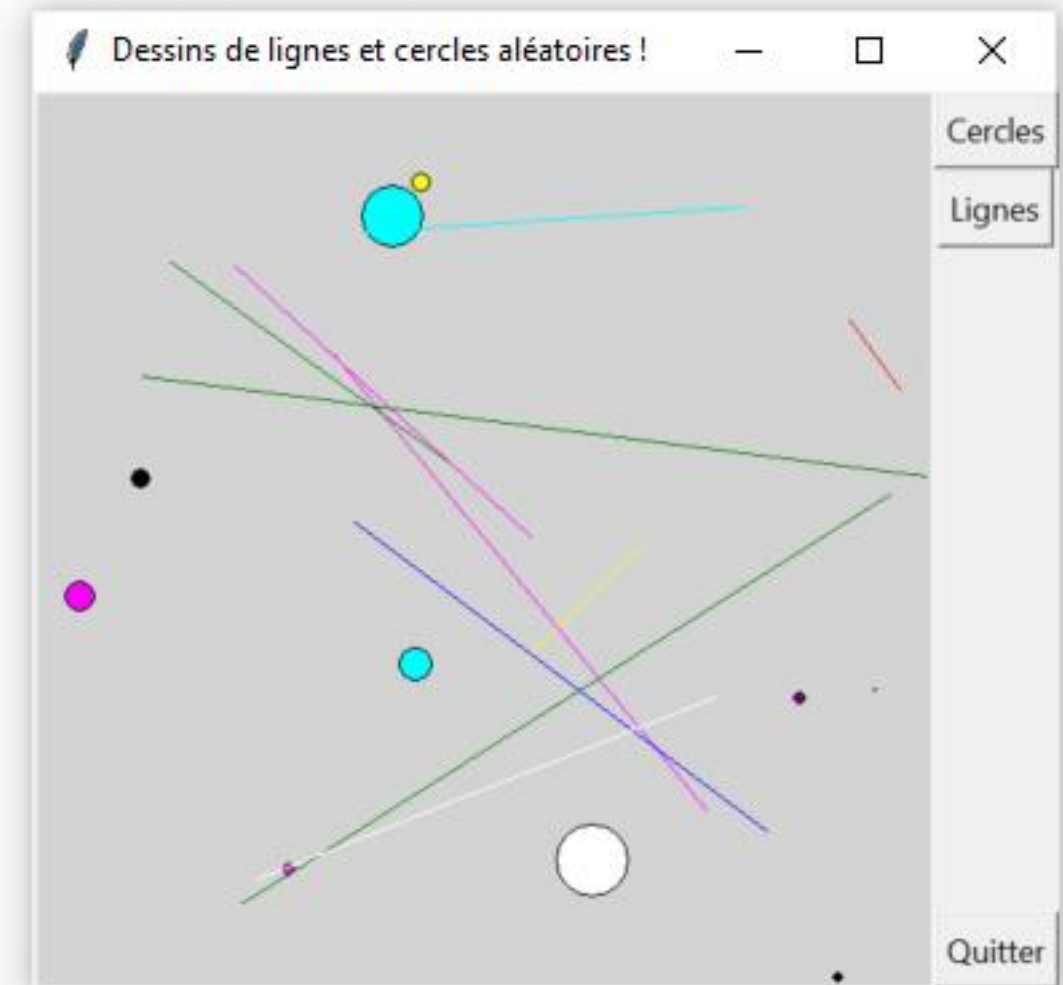
Dessiner avec Tkinter: la classe Canvas (1)

```
1 #dessinTkinter.py
2 import tkinter as tk
3
4 racine = tk.Tk()
5 canv = tk.Canvas(racine, bg="white", height=200, width=200) #Canvas ( master, option=value, ... )
6 canv.pack()
7 canv.create_oval(10, 10, 190, 190, outline="red", width=4) #canv.create_oval(x0, y0, x1, y1, options)
8 canv.create_line(0, 0, 200, 200, fill="black", width=10) #canv.create_line(x0, y0, x1, y1, ..., xn, yn, options)
9 canv.create_line(0, 200, 200, 0, fill="black", width=10)
10 racine.mainloop()
```



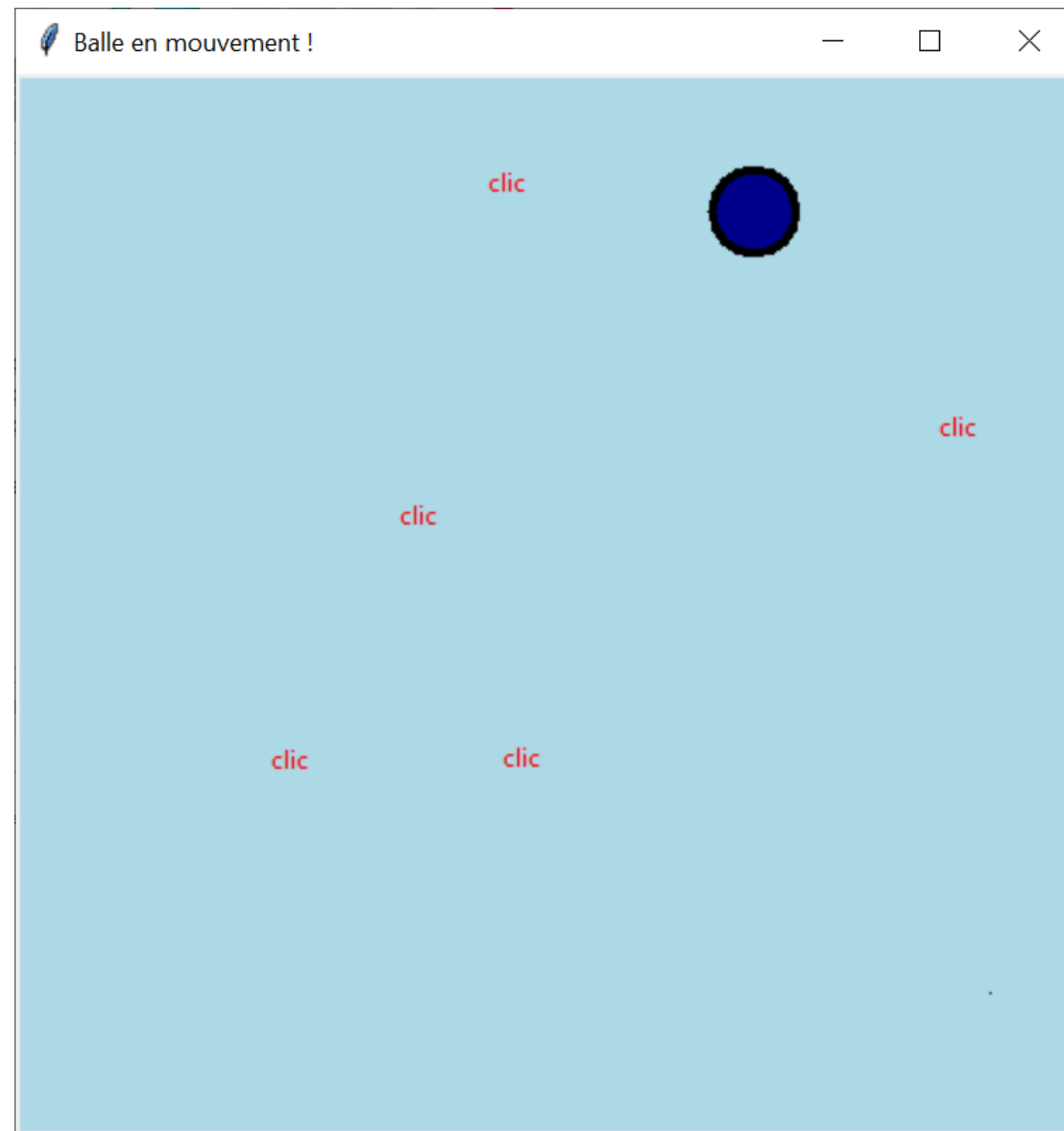
Dessiner avec Tkinter: la classe Canvas (2)

```
1 #dessinClasse.py
2 import tkinter as tk
3 import random as rd # pour générer des coordonnées et couleurs aléatoires
4
5 class AppliCanevas(tk.Tk):
6     taille = 500
7     couleurs = ["black", "red", "green", "blue", "yellow", "magenta", "cyan", "white", "purple"]
8     diametreMax = 50
9     maxDessins = 10
10     def __init__(self):
11         tk.Tk.__init__(self)
12         self.size = AppliCanevas.taille # taille de mon application
13         self.creer_widgets()
14
15     def creer_widgets(self):
16         # création du canevas
17         self.canv = tk.Canvas(self, bg="light gray", height=self.size, width=self.size)
18         self.canv.pack(side=tk.LEFT) #TOP (défaut), BOTTOM, ou RIGHT
19         # boutons
20         self.bouton_cercles = tk.Button(self, text="Cercles", command=self.dessine_cercles) #callback
21         self.bouton_cercles.pack(side=tk.TOP)
22         self.bouton_lignes = tk.Button(self, text="Lignes", command=self.dessine_lignes) #callback
23         self.bouton_lignes.pack()
24         self.bouton_quitter = tk.Button(self, text="Quitter", command=self.quit) #callback
25         self.bouton_quitter.pack(side=tk.BOTTOM)
26
27     def couleur_aleatoire(self):
28         return rd.choice(AppliCanevas.couleurs)
29
30     def dessine_cercles(self):
31         for i in range(AppliCanevas.maxDessins):
32             # génère une coordonnée (deux nombres) aléatoire
33             x, y = [rd.randint(1, self.size) for j in range(2)]
34             diametre = rd.randint(1, AppliCanevas.diametreMax)
35             self.canv.create_oval(x, y, x+diametre, y+diametre, fill=self.couleur_aleatoire())
36
37     def dessine_lignes(self):
38         for i in range(AppliCanevas.maxDessins):
39             # génère une liste de deux coordonnées (quatre nombres) aléatoires
40             x, y, x2, y2 = [rd.randint(1, self.size) for j in range(4)]
41             self.canv.create_line(x, y, x2, y2, fill=self.couleur_aleatoire())
42
43 if __name__ == "__main__":
44     app = AppliCanevas()
45     app.title("Dessins de lignes et cercles aléatoires !")
46     app.mainloop()
```



Dessiner avec Tkinter: la classe Canvas (3)

```
1 # balleMouvementTkinter.py
2 # clic gauche: agrandir la baballe
3 # clic droit: rapetisser la baballe
4 # clic central: relancer la baballe aléatoirement à ce point
5 # touche Tab: quitte l'appli baballe
```



Note: jeter un coup d'oeil à la méthode **after**.

D'autres widgets

- **Checkbutton** : affiche des cases à cocher.
- **Entry** : demande à l'utilisateur de saisir une valeur / une phrase.
- **Listbox** : affiche une liste d'options à choisir .
- **Radiobutton** : implémente des « boutons radio ».
- **Menubutton** et **Menu** : affiche des menus déroulants.
- **Message** : affiche un message sur plusieurs lignes (extensions du widget Label).
- **Scale** : affiche une règle graduée pour que l'utilisateur choisisse parmi une échelle de valeurs.
- **Scrollbar** : affiche des ascenseurs (horizontaux et verticaux).
- **Text** : crée une zone de texte dans lequel l'utilisateur peut saisir un texte sur plusieurs lignes.
- **Spinbox** : sélectionne une valeur parmi une liste de valeurs.
- **tkMessageBox** : affiche une boîte avec un message

Il existe également une extension de Tkinter nommée **ttk**, réimplémentant la plupart des widgets de base de Tkinter et qui en propose de nouveaux (**Combobox**, **Notebook**, **Progressbar**, **Separator**, **Sizegrip** et **Treeview**). Typiquement, si vous utilisez ttk, il est conseillé d'utiliser les widgets ttk en priorité, et pour ceux qui n'existent pas dans ttk, ceux de Tkinter (comme Canvas qui n'existe que dans Tkinter). Vous pouvez importer le sous-module ttk de cette manière : **import tkinter.ttk as ttk**

Références supplémentaires

En français: [livre](#)

En anglais : [exemples](#), [livre](#), [vidéos](#)